Spoke-to-Spoke Lab — Validation Report

Summary

This report validates the spoke-to-spoke traffic hairpinning problem through a VPN gateway and evaluates two fix approaches:

1. **Direct Peering** — Remove forced tunneling UDR, disable gateway transit, add spoke-to-spoke VNet peering ✅
2. **Adjacent Private Endpoint** — Place private endpoints in the consumer's VNet so traffic stays local ✅

Test environment: Azure hub-and-spoke lab in `rg-spoke-to-spoke-lab` (centralus)
Test workload: 1 GB file upload/download cycles between `vm-dbrx` (Spoke 1) and ADLS Gen2 private endpoint (Spoke 2) using azcopy
Test duration: 15 minutes per configuration, 5-minute gaps between tests
Date: 2026-04-06 (all times CDT)

Dashboard Overview — All Three Test Runs

The screenshot below shows the complete Grafana dashboard across all three test configurations in a single view. Each vertical dashed line is a **Grafana annotation** marking a test boundary (begin or end), posted programmatically via the Grafana API using `scripts/grafana-annotate.ps1`. The annotations create clear visual markers that correlate events across all six panels simultaneously.

Reading left to right, the three traffic bursts correspond to:

1. **Broken state** (~13:05–13:21 UTC) — Gateway flows spike to 3.1K, all traffic hairpins
2. **Direct Peering** (~13:29–13:44 UTC) — Gateway drops to baseline, traffic flows peer-to-peer
3. **Adjacent PE** (~13:54–14:09 UTC) — Gateway remains flat, storage data stays in consumer spoke

Each panel includes **min, max, and mean** values in the legend table for quantitative comparison. The gateway panels use a **fixed Y-axis (0–3,500)** so the flat baseline in Fix 2 is visually distinct from the broken state's spike — without fixed scaling, auto-scale would make the ~600 ancillary flows appear as volatile spikes.

The following sections break down each configuration in detail.

The Problem: VPN Gateway Hairpin

> **Note on gateway type**: This lab uses a VPN gateway (VpnGw1), but the same hairpin behavior occurs with an **ExpressRoute gateway**. Both gateway types support `allowGatewayTransit` / `useRemoteGateways` on VNet peerings, and both will process spoke-to-spoke traffic when a catch-all UDR forces `0.0.0.0/0 → VirtualNetworkGateway`. The fixes demonstrated here — direct peering and adjacent private endpoints — apply equally to ExpressRoute environments.

Architecture (Broken State)

```
 Spoke 1 (vm-dbrx)          Hub (vnet-hub)          Spoke 2 (ADLS PE)
10.101.0.0/16               10.100.0.0/16           10.102.0.0/16
     │                          │                       │
     │  UDR: 0.0.0.0/0          │                       │
     │  → VirtualNetworkGateway │                       │
     │                          │                       │
     └──── peering ──────► VPN Gateway ◄──── peering ────┘
            (useRemoteGw=true)    │    (allowGwTransit=true)
                                  │
```

```
                                ALL spoke-to-spoke
                                traffic hairpins here
```

User-Defined Routes (UDRs) on both spokes force a default route (`0.0.0.0/0 → VirtualNetworkGateway`). Combined with `useRemoteGateways: true` on spoke peerings and `allowGatewayTransit: true` on hub peerings, **all** spoke-to-spoke traffic is forced through the VPN gateway — even though the spokes are in the same region.

Effective Routes (Broken)

```
 Source    State    Address Prefix    Next Hop Type
 ________  _______  _________________ ___________________________
 Default   Active   10.101.0.0/16     VnetLocal
 Default   Active   10.100.0.0/16     VNetPeering
 User      Active   0.0.0.0/0         VirtualNetworkGateway   ◄── Forces ALL traffic through gateway
 User      Active   68.47.19.27/32    Internet
 Default   Invalid  0.0.0.0/0         Internet                ◄── Overridden by UDR
```

Key observation: No route for `10.102.0.0/16` (Spoke 2). Traffic to the ADLS private endpoint (`10.102.2.4`) falls under the `0.0.0.0/0 → VirtualNetworkGateway` catch-all.

Grafana — Broken State (08:05–08:21 CDT)

Metric	Min	Max	Mean
Gateway Inbound Flows	601	3,120	1,730
Gateway Outbound Flows	601	3,120	1,730
VM Network Out	378 kB	20.2 GB	11.9 GB
VM Network In	218 kB	20.6 GB	12.0 GB
Storage Ingress	0 B	20.4 GB	11.8 GB
Storage Egress	0 B	40.8 GB	23.6 GB

Gateway flows ramped from **601 to 3,120** — all storage traffic hairpins through the VPN gateway.

Fix 1: Direct Spoke-to-Spoke Peering

Changes Applied

Change	Before (Broken)	After (Direct Peering)
Route tables	`0.0.0.0/0 → VirtualNetworkGateway`	Route **removed**
Hub→Spoke peering	`allowGatewayTransit: true`	`allowGatewayTransit: false`
Spoke→Hub peering	`useRemoteGateways: true`	`useRemoteGateways: false`
Spoke↔Spoke peering	None	**Direct peering** vnet-spoke-dbrx ↔ vnet-spoke-adls

Architecture (Direct Peering)

```
 Spoke 1 (vm-dbrx)          Hub (vnet-hub)          Spoke 2 (ADLS PE)
10.101.0.0/16               10.100.0.0/16           10.102.0.0/16
      |                          |                       |
      |  No default UDR          |                       |
```

```
│                          │                          │
├──── peering ────────────┤──── peering ──────────┤
│  (no gateway transit)    │  (no gateway transit)  │
│                                                   │
└──────── direct peering ──────────────────────────┘
          Traffic goes HERE now (bypasses gateway)
```

Effective Routes (Direct Peering)

```
 Source    State    Address Prefix    Next Hop Type
 ────────  ───────  ────────────────  ──────────────────────
 Default   Active   10.101.0.0/16     VnetLocal
 Default   Active   10.100.0.0/16     VNetPeering
 Default   Active   10.102.0.0/16     VNetPeering            ◄── Direct route to Spoke 2
 Default   Active   0.0.0.0/0         Internet               ◄── Default (no forced tunneling)
 User      Active   68.47.19.27/32    Internet
 Default   Active   10.102.2.4/32     InterfaceEndpoint      ◄── PE routes via peering
 Default   Active   10.102.2.5/32     InterfaceEndpoint
```

Grafana — Direct Peering (08:29–08:44 CDT)

Metric	Min	Max	Mean
Gateway Inbound Flows	600	3,100	1,350
Gateway Outbound Flows	600	3,100	1,350
VM Network Out	385 kB	20.1 GB	12.1 GB
VM Network In	217 kB	20.6 GB	12.2 GB
Storage Ingress	0 B	20.0 GB	12.0 GB
Storage Egress	0 B	40.8 GB	24.1 GB

Gateway flows **dropped from 3,100 to 600** (baseline) within the first 5 minutes. The high initial value is residual from the broken test. Once direct peering is active, the gateway is idle.

Fix 2: Adjacent Private Endpoint

Concept

Place private endpoints for the storage account **in the consumer's VNet** (vnet-spoke-dbrx). The VM connects to a local PE IP (`10.101.2.x`) instead of the remote PE (`10.102.2.x`). Azure creates `/32 InterfaceEndpoint` routes that are more specific than the `0.0.0.0/0` UDR, completely bypassing the forced tunneling path for storage data.

Key advantage: No changes to route tables, peering, or gateway transit. The existing "broken" routing remains intact, but storage traffic stays local.

Note: The catch-all UDR (`0.0.0.0/0 → VirtualNetworkGateway`) remains active, so ancillary traffic (AAD authentication, DNS, etc.) still transits the gateway. Only storage data is redirected via the `/32` routes.

Changes Applied

Change	Before (Broken)	After (Adjacent PE)
Route tables	`0.0.0.0/0 → VirtualNetworkGateway`	**Unchanged**

Change	Before (Broken)	After (Adjacent PE)
Hub↔Spoke peering	`allowGatewayTransit: true`	**Unchanged**
Spoke→Hub peering	`useRemoteGateways: true`	**Unchanged**
vnet-spoke-dbrx subnets	`subnet-dbrx` only	Added `subnet-pe` (10.101.2.0/24)
Private endpoints	DFS + Blob PEs in spoke-adls only	**Added** DFS + Blob PEs in spoke-dbrx

Architecture (Adjacent PE)

```
 Spoke 1 (vm-dbrx)          Hub (vnet-hub)          Spoke 2 (ADLS PE)
10.101.0.0/16               10.100.0.0/16           10.102.0.0/16
     │                           │                       │
     │  UDR: 0.0.0.0/0          │                       │
     │  → VirtualNetworkGateway  │                       │
     │  (STILL ACTIVE)           │                       │
     ├──── peering ──────► VPN Gateway ◄──── peering ────┤
     │                                                   │
     │  ┌─────────────────┐                              │
     │  │ subnet-pe       │                              │
     │  │ 10.101.2.0/24   │                              │
     │  │                 │                              │
     │  │ pe-adls-dfs-local  ─── Azure backbone ──► ADLS │
     │  │ pe-adls-blob-local ─── Azure backbone ──► ADLS │
     │  └─────────────────┘                              │
     │                                                   │
     └── VM traffic goes to LOCAL PE (bypasses gateway) ──┘
```

Effective Routes (Adjacent PE)

```
 Source    State    Address Prefix    Next Hop Type
────────  ───────  ────────────────  ──────────────────────
Default   Active   10.101.0.0/16     VnetLocal
Default   Active   10.100.0.0/16     VNetPeering
User      Active   0.0.0.0/0         VirtualNetworkGateway   ◄── UDR STILL ACTIVE
User      Active   68.47.19.27/32    Internet
Default   Invalid  0.0.0.0/0         Internet
Default   Active   10.101.2.4/32     InterfaceEndpoint       ◄── Local DFS PE (/32 overrides UDR)
Default   Active   10.101.2.5/32     InterfaceEndpoint       ◄── Local Blob PE (/32 overrides UDR)
```

Key observation: The `/32 InterfaceEndpoint` routes at `10.101.2.4` and `10.101.2.5` override the catch-all UDR. Storage traffic stays within vnet-spoke-dbrx.

Grafana — Adjacent PE (08:54–09:09 CDT)

Metric	Min	Max	Mean
Gateway Inbound Flows	588	637	615
Gateway Outbound Flows	588	637	615

Metric	Min	Max	Mean
VM Network Out	454 kB	19.5 GB	11.9 GB
VM Network In	266 kB	20.3 GB	11.9 GB
Storage Ingress	0 B	19.3 GB	11.8 GB
Storage Egress	0 B	40.5 GB	23.6 GB

Gateway flows remained **essentially flat at 588–637** — the gateway is not processing storage data. The slight variation (~50 flow range) is ancillary traffic (AAD auth, DNS queries) still caught by the catch-all UDR.

Conclusion

Comparison of All Configurations

Metric	Broken (Hairpin)	Fix 1 (Direct Peering)	Fix 2 (Adjacent PE)
Status	⚠️ Working (inefficient)	✅ Working	✅ Working

Metric	Broken (Hairpin)	Fix 1 (Direct Peering)	Fix 2 (Adjacent PE)
Gateway Flows (Max)	**3,120**	600 (baseline)	**637** (flat baseline)
Gateway in data path?	Yes (all traffic)	**No**	**No** (storage data bypassed)
VM Network Out (Max)	20.2 GB	20.1 GB	19.5 GB
Storage Egress (Max)	40.8 GB	40.8 GB	40.5 GB
Routing changes	—	UDR removed, gateway transit disabled	**None**
Peering changes	—	Spoke-to-spoke peering added	**None**
Infrastructure added	—	None	PE subnet + 2 PEs

Fix 1: Direct Peering

- Removes the gateway from the data path entirely by fixing the routing architecture
- Requires changes to route tables, peering settings, and adding spoke-to-spoke peering
- Best when you want a clean network architecture without forced tunneling

Fix 2: Adjacent Private Endpoint (Recommended)

- Bypasses the gateway without changing any routing or peering settings
- The forced tunneling UDR remains active, but `/32 InterfaceEndpoint` routes override it
- Minimally invasive — only adds a PE subnet and two private endpoints in the consumer's VNet
- Best when you can't change the existing network architecture (e.g., shared hub managed by a central team)
- **Note**: Ancillary traffic (AAD auth, DNS, etc.) still transits the gateway due to the catch-all UDR — only storage data is bypassed

Other Approaches (Not Tested)

Azure Virtual WAN (VWAN) would also address this issue. VWAN's hub provides native spoke-to-spoke routing without requiring manual peering or UDRs — the VWAN hub router automatically learns spoke prefixes and forwards traffic directly between spokes. In a VWAN topology, spoke-to-spoke traffic does not hairpin through the VPN or ExpressRoute gateway; it is routed through the VWAN hub router at no additional gateway cost. VWAN also supports routing intent and policies that provide more granular control over inter-spoke traffic flows. However, VWAN requires migrating from a traditional hub-and-spoke architecture and was not tested in this lab.

Security and Compliance Implications

Each fix has different security posture trade-offs. Organizations subject to regulatory controls (e.g., PCI-DSS, HIPAA, FedRAMP) should evaluate these carefully.

Broken State (Baseline)

- **Forced tunneling**: All traffic (`0.0.0.0/0`) routes through the gateway, providing a **central inspection point** — this is often a compliance requirement for environments mandating egress filtering or network-level DLP
- **Gateway as chokepoint**: The gateway sees all flows, enabling logging, IDS/IPS integration, and traffic analytics
- **Risk**: Gateway saturation degrades availability — PPS and throughput limits can cause packet drops, impacting SLA compliance

Fix 1: Direct Peering

- **Removes forced tunneling entirely** — spoke-to-spoke and internet traffic no longer transits the gateway
- **No central inspection point**: Traffic between spokes bypasses the hub completely. If your compliance framework requires all inter-VNet traffic to pass through a firewall or NVA for inspection, this fix **violates that requirement**
- **Mitigation**: Deploy an Azure Firewall or NVA in the hub and route spoke-to-spoke traffic through it instead of the gateway. This was not implemented in this lab.
- **NSG enforcement**: Without forced tunneling, NSGs on spoke subnets become the primary network-level control — ensure they are configured to restrict lateral movement between spokes
- **Private endpoint security**: Data still flows over private endpoints (no public internet exposure), maintaining data-in-transit confidentiality

Fix 2: Adjacent Private Endpoint

- **Preserves forced tunneling** — the catch-all UDR remains active, maintaining the central inspection/logging posture for all non-PE traffic
- **Storage data bypasses the gateway** via `/32 InterfaceEndpoint` routes, but stays entirely within the Azure backbone (PE-to-PE, never touches the internet)
- **Ancillary traffic still inspected**: AAD auth, DNS, management traffic continues through the gateway — compliance logging for authentication flows is maintained
- **Private Link compliance**: Data plane traffic uses Azure Private Link, which satisfies most data-sovereignty and data-in-transit requirements (encrypted within Azure backbone)
- **Additional PE surface area**: Two new private endpoints in the consumer VNet mean additional resources to govern — ensure PE approval workflows, DNS zone policies, and RBAC are applied consistently
- **Best compliance fit**: For environments requiring forced tunneling for audit/inspection purposes, adjacent PE is the **least disruptive** fix because it preserves the existing network security architecture

Summary

Concern	Broken	Fix 1 (Direct Peering)	Fix 2 (Adjacent PE)
Forced tunneling preserved	✅ Yes	❌ **Removed**	✅ Yes
Central inspection point	✅ Gateway sees all traffic	❌ Gateway bypassed	⚠️ Gateway sees ancillary traffic
Data stays on private network	✅ PE traffic	✅ PE traffic	✅ PE traffic
Gateway saturation risk	⚠️ **High**	✅ None	✅ None (storage data bypassed)
NSG as primary control	⚠️ UDR overrides	✅ Required	⚠️ UDR still active
Compliance change required	—	⚠️ May violate forced tunneling mandate	✅ Minimal

Bicep Configurations

All three states are codified as self-contained Bicep deployments:

- **`bicep/lab-current/`** — Reproduces the broken hairpin state
- **`bicep/lab-fixed-direct-peering/`** — Fix 1: Direct spoke-to-spoke peering
- **`bicep/lab-fixed-adjacent-pe/`** — Fix 2: Adjacent private endpoints in consumer VNet

Deploy any configuration with:

```
az deployment group create \
  --resource-group rg-spoke-to-spoke-lab \
  --template-file bicep/<config>/main.bicep \
  --parameters adminUsername=<user> adminPublicKey='<key>' \
               allowedSshSourceIp=<ip> storageAccountSuffix=<suffix>
```